

Data Collection & Extraction Architecture

Project Presentation — Automated Dynamic Multi-Modal Data Ingestion Pipeline

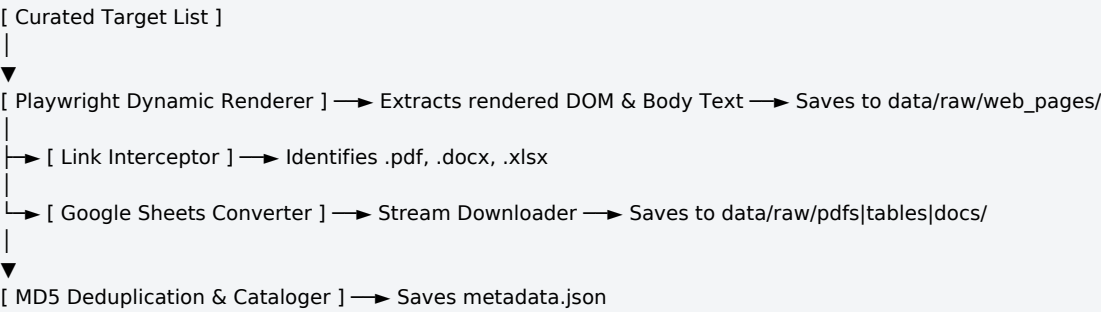
1. Executive Summary / Problem Statement

When scraping academic institution websites such as IIIT Nagpur, standard static web crawlers (requests + BeautifulSoup) can fail for two major reasons:

- 1. Single Page Application (SPA) Rendering: The website uses a modern React / Vite frontend stack. An initial HTTP request may return a bare HTML shell (for example, `<div id="root"></div>`) while the actual text, links, and tables are injected client-side through JavaScript.
- 2. Scattered Multi-Source Content: Official information is distributed across dynamic notice boards, attached PDFs, DOCX/XLSX documents, and embedded Google Spreadsheets (published HTML links for fee structures and timetables).

To solve this, we engineered an automated, dynamic multi-modal data ingestion pipeline that renders pages, discovers linked assets, converts spreadsheet wrappers into downloadable data, deduplicates files, and maintains provenance metadata.

2. Technical Stack & Pipeline Workflow



3. Detailed Component Breakdown

A. Targeted Scope vs. Blind Crawling (Cells 2 & 3)

Why targeted URL lists? Unrestricted recursive crawling can get stuck in infinite query loops, calendar widgets, or external redirect links.

Our approach: We curated an explicit list of 72 high-priority URLs across 8 core domain categories:

Category	Primary Content
Academics	Syllabus, Timetables, Courses
Admissions	B.Tech, PG, Ph.D., Fees, Scholarships
Placements	Statistics, Companies, Internships
Governance	Administration, Committees
Notices	Tenders, Circulars, Official Documents
Research	Projects, Publications, Patents
Student Affairs	Hostel Life, Mess, Clubs
Institute	Infrastructure, Profile, NIRF

B. Dynamic Rendering Engine (Cells 1, 6 & 7)

Playwright Headless Browser: A headless Chromium instance navigates to each target URL, executes JavaScript bundles, and waits for the page to reach a suitable loaded state such as networkidle.

Windows Jupyter Concurrency Resolution: To avoid NotImplementedError and event-loop collisions in Windows Jupyter environments, the browser engine runs inside an isolated background thread using asyncio.to_thread(), providing cleaner subprocess management.

C. Asset Discovery & Conversion (Cells 4 & 5)

HTML Text Extraction: Non-content elements such as <script>, <style>, <nav>, and <noscript> are removed before extracting contextual plain text with source-page metadata.

Google Sheets Endpoint Transformation: Fee tables and timetables may appear as Google Sheets published HTML wrappers. The pipeline detects the /pubhtml URL, extracts the gid, and reconstructs a direct CSV endpoint.

Example: .../pubhtml?gid=1271144793 → .../pub?output=csv&gid=1271144793

Streaming Binary Downloader: Large PDFs and other binary assets are retrieved using chunked HTTP streaming (for example, iter_content(chunk_size=8192)) with a 60-second socket timeout, reducing the risk of freezing the browser context.

D. Data Governance & Deduplication (Cells 4 & 6)

MD5 Content Hashing: Before saving a webpage or binary file, the pipeline computes a 128-bit MD5 checksum from the file bytes. If identical content is linked from multiple pages, it is downloaded only once.

This prevents redundant copies—for example, the same fee PDF linked from both an Admissions page and a Notices page is retained as a single asset.

Unified Metadata Ledger (metadata.json): Every ingested asset is recorded with structured provenance and storage metadata.

Field	Purpose
doc_id	Unique hash-derived identifier
title	Extracted title or button anchor text
category	Inferred domain/category
file_type	html, pdf, table, docx, etc.
source_url	Original asset URL
parent_page_url	Page from which the asset was discovered
local_path	Relative filesystem destination

4. Summary Table for Quick Reference

Feature	Problem Solved	Technical Solution Used
React SPA Scrape	Empty <div id="root"> responses	Playwright Chromium Headless Engine
Jupyter Windows Error	NotImplementedError / event-loop crash	Thread-isolated worker via asyncio.to_thread
Embedded Spreadsheets	Browser opens HTML preview instead of raw data	Regex URL converter to forced ?output=csv
Duplicate Files	Same PDF linked on multiple pages	MD5 byte-level hash deduplication
Traceability	Knowing where retrieved context originated	Centralized metadata.json ledger

Architecture outcome: The resulting ingestion layer provides dynamic rendering, multi-format asset discovery, content deduplication, and end-to-end provenance—forming a reliable foundation for downstream document processing, retrieval, indexing, and RAG workflows.